



Draw it or lose it

CS 230 Project Software Design Template
Version 3.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	5

Document Revision History

Version	Date	Author	Comments
1.0	03/22/26	Courtney Jenkins	Added Executive Summary, Requirements, Design Constraints, and Domain Model
2.0	04/05/26	Courtney Jenkins	Added Evaluation for Client, Server, and Development tools for OS
3.0	04/19/26	Courtney Jenkins	Added Recommendations

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room wishes to expand its customer base from an exclusive mobile audience to a web-based platform capable of supporting games for a broader audience. With the expansion of the popular Android app Draw It or Lose It into a web-based, multi-platform game, transitional challenges arise. The challenges are as follows: Maintaining a consistent game state across all the connected systems, ensuring unique entities, and enforcing a single active game instance in memory.

Addressing these challenges can be solved with the construction of an application centered on object-oriented programming principles and design patterns. First, we will use a base Entity class to improve the scalability, encapsulation, and inheritance. The Entity class allows its child classes to inherit base attributes and methods, contained only within the Entity class, to operate the web-based application. As there is a central location for base attributes and methods, the number of instances for each child class can be scaled to support many active users. A Singleton Pattern will be employed to ensure that only one instance of the game service will exist. With only one instance, The Gaming Room can have centralized control over all games and reduce memory consumption. Lastly, we will use an Iterator Pattern to provide uniqueness for each of the games, teams, and players.

Our object-oriented programming approach will keep The Gaming Room's business rules in place, enable scalability for web-based platforms, and maintain the applications architecture in accordance with industry best practices.

Requirements

- (a) Only a single game instance can exist in the memory. (Business and Technical Requirement)
- (b) Game, team, and player names must be unique across the platform. (Business and Technical Requirement)
- (c) The application must be supported across multiple devices and browsers. (Technical Requirement)
- (d) Multiple users must be able to create and join games at the same time. (Technical Requirement)
- (e) The web-based application must be able to handle growth for games. (Technical Requirement)

Design Constraints

- (a) Implementing the Singleton Pattern, we can prevent duplicate instances across the service. For a larger system, additional support may be required to manage these single sessions for many users.
- (b) Utilizing lists to hold active entities and corresponding iterators to parse through each list will allow unique naming throughout the platform. Larger lists could adversely affect system performance.
- (c) Appropriate backend support must be organized to ensure that devices can connect to the web application. Recognize that as clients access the game, many different platforms will connect to the server. Client tracking should be achieved through a centralized ID per session. The GameService class centralizes this logic, and the Entity class ensures reusability. Operation with a larger distributed database can expand these object-oriented principles.
- (d) The Singleton Pattern assures that data will not be corrupted even as multiple instances are accessed or modified. As multiple users create games, the resources employed by each game must be protected of the UX.
- (e) To handle larger audiences, persistent database storage may be required. Relational databases would support larger indices, faster iterators, and improve overall UX.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical

components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

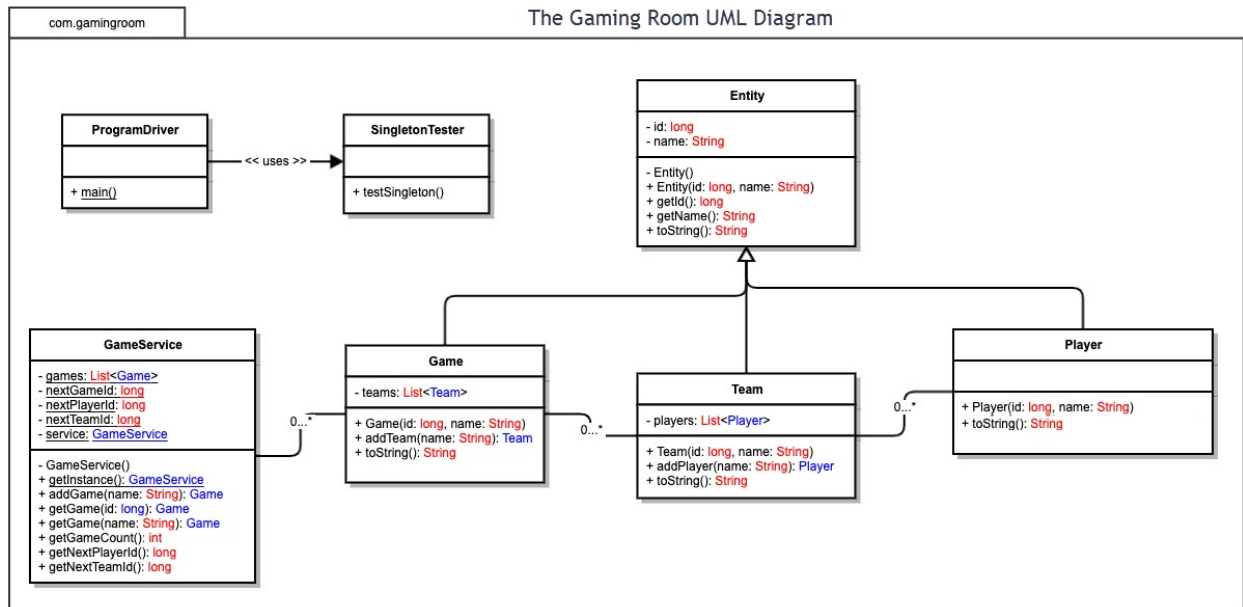
Domain Model

In the following paragraphs, we will discuss the construction of a UML class diagram. Some technical terms we will cover are classes, attributes, methods, and patterns. These are core concepts in object-oriented programming and will help understanding the application. Classes are templates that contain an object's attributes and behaviors. Parent classes can pass on attributes and behaviors to child classes. Attributes are variable, or data fields, that store the properties of an object. Methods are the behaviors that define what the object can do. Patterns are templates for solving software design problems.

The Entity class is a parent class that contains the base attributes for ID and names. It also includes the methods for accessing these attributes (getters). The Game, Team, and Player classes all inherit from the Entity class. The Game class contains a list of active teams, allows adding teams, and has a 0-to-many relationship with the Team class. The relationship implies that a game can contain many teams. The Team class contains a list of active players, allows adding players, and has a 0-to-many relationship with the Player class. The Player class represents a single player for the application and links them to a specific team. The add behaviors use an Iterator Pattern to verify the uniqueness of team and player names. The GameService class contains a list of active games and ID counters and uses the Singleton Pattern. A Singleton Pattern maintains a global access point with a single instance of a game. The ProgramDriver and SingletonTester classes run the main program and perform a Singleton test, respectively.

To summarize the relationships between each class: The GameService class manages the Game class, the Game class contains the Team class, the Team class contains the Player class, and they are all child classes of the parent Entity class.

Object-oriented programming principles on display here are encapsulation, inheritance, abstraction, and polymorphism. Encapsulating data with private or protected modifiers restricts access to or modification of key data, such as IDs, names, and active lists. Maintainability is enhanced, and redundancy is reduced when using inheritance, such as the Entity parent class to its child classes. Abstraction is the process of showing only the essential features of code design and implementation, which makes the code easier to maintain and easier to understand. An example is the relationship between the ProgramDriver (main method) class and the GameService class. GameService facilitates all the interactions between the classes. Polymorphism is an example of many objects calling a method in their own way. For the `getGame()` method, we can see several versions that can take an integer, a long, or a string. Objects can call this method using any of these and execute it based on the object's need.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Pros: • Uses a Unix-based system, so it shares similarities with Linux. • Fully supported with both Apache and Nginx web server software. • It is the only way to make the game functional on iOS devices (app). • Provides a great environment for developing UX/UI experiences. Cons: • Apple hardware has a high up-front cost, and very little support for web server hardware. • Very limited integration tools for web server environments. • macOS cannot be run on non-Apple devices.	Pros: • Highly customizable, open-source operating system that offers flexibility in how servers are maintained and operated. • Since Linux is open-source, there are no license fees. • Well-suited for containerization, which can improve server efficiency and resource management. • Fully supported with both Apache and Nginx web service software. Cons: • Many creative/office suites are not natively supported with Linux • There is a steep learning curve if your office is unfamiliar with Linux	Pros: • Most familiar to people, so there could be a small learning curve for server operations. • It has many natively supported creative/office suites. • It is already integrated with the .NET ecosystem, so there is a diverse library to aid developers. • Strong enterprise support options to help reduce downtime and data loss. Cons: • Scaling usage can be costly with Windows due to licensing fees. • Requires more memory as compared to Linux, and would contribute to a higher cost for hardware. • It is not as widely used for cloud and web server development.	Pros: • None for web server environments. Cons: • No real way to host a server with a mobile platform.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Client Side	OS has availability to run HTML5 in web-browsers (see below).	OS has availability to run HTML5 in web-browsers (see below).	OS has availability to run HTML5 in web-browsers (see below).	OS has availability to run HTML5 in web-browsers (see below). Possibility for application development, but overall easier to access for mobile users.
Development Tools	OS will support all the necessary IDEs and languages for development (see below). In addition, access to Xcode for iOS applications.	OS will support all the necessary IDEs and languages for development (see below). Alternatively, you could work within Eclipse.	OS will support all the necessary IDEs and languages for development (see below). Alternatively, you could work within Eclipse.	Development for web-based application will not occur within this OS

With all these factors in mind, I recommend running the server-side web app on a Linux system. Linux offers many tools, libraries, and support for server-based deployment. Not to mention, overwhelming industry use in cloud and web server environments. All these factors aid in the efficiency, maintenance, and scalability of the web app. The reduced licensing and hardware costs make it an attractive alternative to other operating systems. Associated server fees would scale with the user base as the application becomes more popular.

Client Side

To support multiple clients, we recommend using HTML5 and Responsive Web Design (RWD) to ensure compatibility across all browsers. HTML5 can be run on all browsers without requiring any extra installations for users and supports APIs, audio, memory, etc. RWD is an architecture that aims to deliver fluid, size-appropriate UIs across all screens.

Using a web-based application has a specific cost advantage by eliminating the need to develop a mobile app. Costs would occur from the purchase and use of “test specimens” to ensure efficient operation for older devices. To build a backend that can support a large number of users at once, additional fees could be incurred as server usage and load balancers scale.

Planning, development, testing, and maintenance times will be compressed due to the creation of a single, web-based application. However, there would still be a large lead to bury in testing overhead to ensure it functions properly across all browsers. Timing and drawing rendering are essential to the game’s operation, so these would need to be extensively developed to ensure fluidity across all platforms.

An appropriate workforce for the job would include frontend developers, backend developers, and platform engineers. The frontend engineers would be responsible for building a website using the HTML5 framework and adhering to RWD architecture. Backend engineers are responsible for shoring up the API and server availability. Platform or DevOps engineers are responsible for implementing continuous integration/deployment (CI/CD) practices to ensure proper deployment and scaling for the

web application. You may want to consider hiring additional mobile app development experts, especially for iOS.

Development

The development team should use Visual Studio (VS) as its IDE. VS is a collaborative IDE with strong project management features that offer advanced debugging tools, code suggestions, and works as a comprehensive framework for a wide range of languages (ASP.NET, ASP.NET Core, Node.js, React, Angular, Vue.js, JavaScript, TypeScript, HTML, and CSS). JavaScript (or TypeScript) can handle both frontend and backend needs, with utility in HTML and Node.js. JavaScript allows developers to access both libraries (utilizing coding tools as necessary) and frameworks (customizing a pre-formatted structure) to tackle the task of a web-based application. Using Docker would allow containerization, thereby easing deployment and scaling throughout the development life cycle. GitHub is a useful repository that allows your team to manage revisions and control application development. Multiple teams can prove useful in this scenario. You could divvy up work between the frontend and backend teams, which could help with scaling and rapid deployment. In this scenario, smaller teams could overlap functions to aid in development. If the team is experienced and familiar with the IDE/languages, a full-stack can be deployed, with a DevOps engineer serving as a manager throughout the development lifecycle. Most of the tools are open-source, so they are free and accessible.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. Operating Platform:

For a multi-user, web-based application, I recommend using a Linux-based server operating system. Linux is an open-source software that caters to superior stability, scalability, and efficiency. Linux has native support for Docker and Kubernetes, allowing for consistency throughout the server architecture and life cycle of the application. Many cloud services (i.e., AWS, Azure, and Google Cloud) are compatible with a Linux-based operating system, as well. The customizability of Linux gives the freedom to maintain maximum hardware resources for running Draw It or Lose It's logic. In keeping with The Gaming Room's needs, Linux offers support for many different platforms through a web-based architecture.

2. Operating Systems Architectures:

Linux is a monolithic kernel operating system architecture. This modular architecture maintains a degree of separation between the server (kernel) and the application (user space). The variety of users and their platforms may have issues that could cause errors while playing the game, and the modularization allows for corrective actions to take effect without causing problems for the whole server. In addition, you can load drivers to allow the game application to scale based on the hardware on the user-facing end of the application. For the server-side adjustments, Docker and Kubernetes support allow for superior scaling with the game service.

3. Storage Management:

The web application will need to store user data, gaming assets, and object storage to ensure the best product. Relational databases, such as PostgreSQL and MySQL, are excellent tools for handling all user-associated and gaming data. Local assets for the server should be maintained with the Linux-based file system, ext4. It provides security and efficiency, as well as a stable server environment for Draw It or Lose It. The small file size efficiency and ability to adjust the file system size further enhance the flexibility of our server-based operating system. Using a cloud-based object storage, we can have

cost-effective and scalable storage for the image database. Each of our images contains unique identifiers and metadata that can be used to call upon the image for the game's operation. Ensuring our memory is effectively stored and manageable is essential to enabling future DevOps for Draw It or Lose It.

4. Memory Management:

Linux uses concepts such as virtual memory and chaining that enhance the user experience in real-time. Virtual memory allows the transfer of data between RAM and a hard disk to facilitate running large programs. Small pages are swapped between the two to allow flexibility for multiple users and a large workload on the server. We can use this, as well, to reduce the costs required for the initial start of Draw It or Lose It as a web-based application. Game data will be cached to reduce the latency experienced by users. We can keep the data current by implementing a Least Recently Used (LRU) policy, which eliminates unused or stale data.

5. Distributed Systems and Networks:

The operating system is required to support a large user base on a variety of platforms, so connectivity and consistency are important. We will need to ensure the network environment is curated for latency and to reduce outages. Having a centralized server will be essential to using a client-server model, and therefore facilitating access to Draw It or Lose It to user platforms. We need to use a service that allows RESTful APIs to function, ensuring the platforms interact correctly without a server. Our services need to detect and redirect traffic as necessary to ensure latency issues are minimized. Nginx can operate as the load balancer to help with our traffic scaling and outages. As previously mentioned, Kubernetes allows the application structure to be managed across all variations of the server.

6. Security:

We intend to use strong security features to protect the user and our application data. Users will need to provide authentication and authorization when accessing the game by adhering to OAuth principles. We recommend encryption on all communications with HTTPS utilizing TLS, to help mitigate interception for user data and even game drawings. Firewalls and constant security patching will help shore up any issues with the Linux system. Role-Based Access Control (RBAC) is used, in conjunction with SELinux, to help control the data flow and access. Admins and Managers can have roles that allow them to access core game objects and data, while the user only has access to the data necessary to play the game. Using RBAC keeps the essential data and privileged user data safe from breach if the security is compromised.